

DEATH IS NOT GAME OVER

THE CURIQUINGUE TAMAGOTCHI CIRCUIT — ASSEMBLY MANUAL

SEE IT RUNNING

These clips were recorded running the program that was current on the Arduino at that time:

- youtube.com/shorts/MdQM2oSGQp8
- youtube.com/shorts/F8pmUEmmahM

The code in this manual — saved in the project's GitHub repository as **Demo Code** — is a functional demo that aspires to the full experience shown in those videos and described in the Game Flow below. It is not yet the complete, eight-step cycle end to end. See "About This Demo Code" just below for exactly what it does and doesn't do, and why.

WHAT THIS DEMO IS

The Curiquingue Tamagotchi Circuit is a hand-built Arduino arcade where a pixel-art Andean raptor tries to survive the increasingly polluted Chibunga River, flowing down from Chimborazo's glaciers. In Kichwa tradition, its rivers and springs are inhabited by elemental beings that cannot exist where water is contaminated — in strictly scientific terms, contaminated water cannot sustain life. The curiquingue, understood in Andean tradition as a messenger between earth and sky, becomes here a real indicator of environmental health, surpassing its folkloric value.

It descends from its páramo home to dance in Riobamba's Pase del Niño, carrying that journey into the river below. Struck too many times, it passes into Uku Pacha, searching a maze for clean water — then rises into Hanan Pacha, whole again, its light falling back as rain.

ABOUT THIS DEMO CODE — READ THIS FIRST

For technology-agnostic readers: the Arduino Uno is a small, inexpensive microcontroller board with very little onboard memory — only 2KB of RAM, shared between everything the program needs to keep track of while running. That is not enough space to hold the entire eight-step Game Flow described above as one continuous program.

Because of that, the Demo Code in this manual runs only part of the full cycle: the mountain descent scene (Kay Pacha), the maze (Uku Pacha), and a handful of the transition animations between them — not the complete loop through Hanan Pacha and back. It is a first, working approach, not the finished game.

The next step is migrating this same design to an ESP32 microcontroller, which has roughly 250 times more RAM (520KB) — enough to run the entire Game Flow as a single, uninterrupted program, exactly as described above and shown in the videos.

GAME FLOW

1. Splash — "Curiqingue Tamagotchi" fades in, line by line.
 2. Instructions.
 3. Kay Pacha, intro — the curiqingue walks in place.
 4. Kay Pacha, the descent — descending 3000 m.a.s.l. from the Chimborazo volcano basin, down into the Riobamba valley, to dance at the Pase del Niño. Hazards begin; three hits, or the full distance survived, trigger the threshold.
 5. Uku Pacha — a red strobe entry, then the maze, centered on screen, in search of the pogyo (Kichwa for water spring).
 6. Hanan Pacha, ascend — the curiqingue rises, whole again.
 7. Hanan Pacha, the Andean glyph rain — chakana, spiral, inti, and the rest fall before the cycle turns over.
 8. "Death is not game over" — held, then the loop returns to step 3.
-

INTRODUCTION TO THE CURIQUINGUE TAMAGOTCHI CIRCUIT

Ever wondered how a small screen and a handful of lights can tell a whole story? This project introduces you to the Curiqingue Tamagotchi Circuit — a hand-built Arduino arcade where a pixel-art Andean raptor descends from the mountains, faces a polluted river, and passes through a threshold instead of simply "losing." In this build, we're using an Arduino Uno, a small OLED display, an analog joystick, and a strip of addressable NeoPixel LEDs — all parts that are affordable and beginner-friendly, even if the story they tell is not simple at all.

KEY COMPONENTS

1. Arduino Uno — controls everything: reads the joystick, draws the screen, and drives the lights.
2. Analog Joystick — lets the player move the curiqingue and jump.
3. SSD1306 OLED Display (128×64) — shows the game itself.
4. 22-LED NeoPixel Strip — lights that change color depending on what is happening in the game.
5. Jumper wires — for making all the connections.
6. Breadboard — to hold everything together without soldering, while prototyping.

OPERATION

In this circuit:

- The joystick works as a movement controller, the same way a potentiometer does:
 - Moving it left/right changes a voltage on the X axis (0–5V)
 - Moving it up/down changes a voltage on the Y axis (0–5V)
 - Arduino reads both as values from 0–1023
- The OLED shows whichever scene the game is currently in — the descent, the maze, or one of the threshold moments.
- The NeoPixel strip is updated on every loop, and always reflects the current game state — it is never just decorative lighting.
- Arduino redraws the whole screen every loop — there are no saved frames, everything is calculated live.

REQUIRED MATERIALS

1. Arduino Uno

- USB cable / USB-C adapter, if needed
2. Analog Joystick module
 3. SSD1306 OLED Display, 128×64, I2C
 4. NeoPixel Strip, 22 LEDs (WS2812B)
 5. Jumper wires
 6. Breadboard

COMPONENTS

Electronic Component	What it does
Arduino Uno	The board that runs the whole game — reads the joystick, draws the screen, drives the lights.
Analog Joystick module	Two potentiometers (X/Y) plus a click button. Moves the curiquestue and triggers jumps.
SSD1306 OLED display (128×64, I2C)	The screen. Shows the descent, the maze, and every threshold scene.
22-LED NeoPixel strip (WS2812B)	Addressable RGB LEDs. Never just decoration — always tied to the current game phase.

TOOLS NEEDED

- Wire strippers
- Computer with Arduino IDE installed

STEP-BY-STEP ASSEMBLY GUIDE

SAFETY FIRST

- Always disconnect power before making connections
- Avoid touching electronic components directly
- Work on a clean, non-metallic surface

ASSEMBLY STEPS

Part 1: Initial Setup

1. Place your Arduino on the left side of the breadboard
2. Make sure your workspace is clear and well-lit

Part 2: Joystick Connection

1. Identify the joystick pins: GND, +5V, VRX, VRY, SW
2. Connect the joystick wires:

Connect GND from the joystick to Arduino's GND pin

Connect +5V from the joystick to Arduino's 5V pin

Connect VRX from the joystick to Arduino's A0 pin

Connect VRY from the joystick to Arduino's A1 pin

Note: the click button (SW) is wired but not used for jumping — jumping is read from the Y axis instead, moving the stick up. This was a deliberate change after testing the SW pin's behavior on this board.

Part 3: OLED Display Connection

1. Identify the OLED pins: VCC, GND, SCL, SDA
2. Connect the OLED wires:

Connect VCC from the OLED to Arduino's 5V pin

Connect GND from the OLED to Arduino's GND pin

Connect SCL from the OLED to Arduino's A5 pin

Connect SDA from the OLED to Arduino's A4 pin

Part 4: NeoPixel Strip Connection

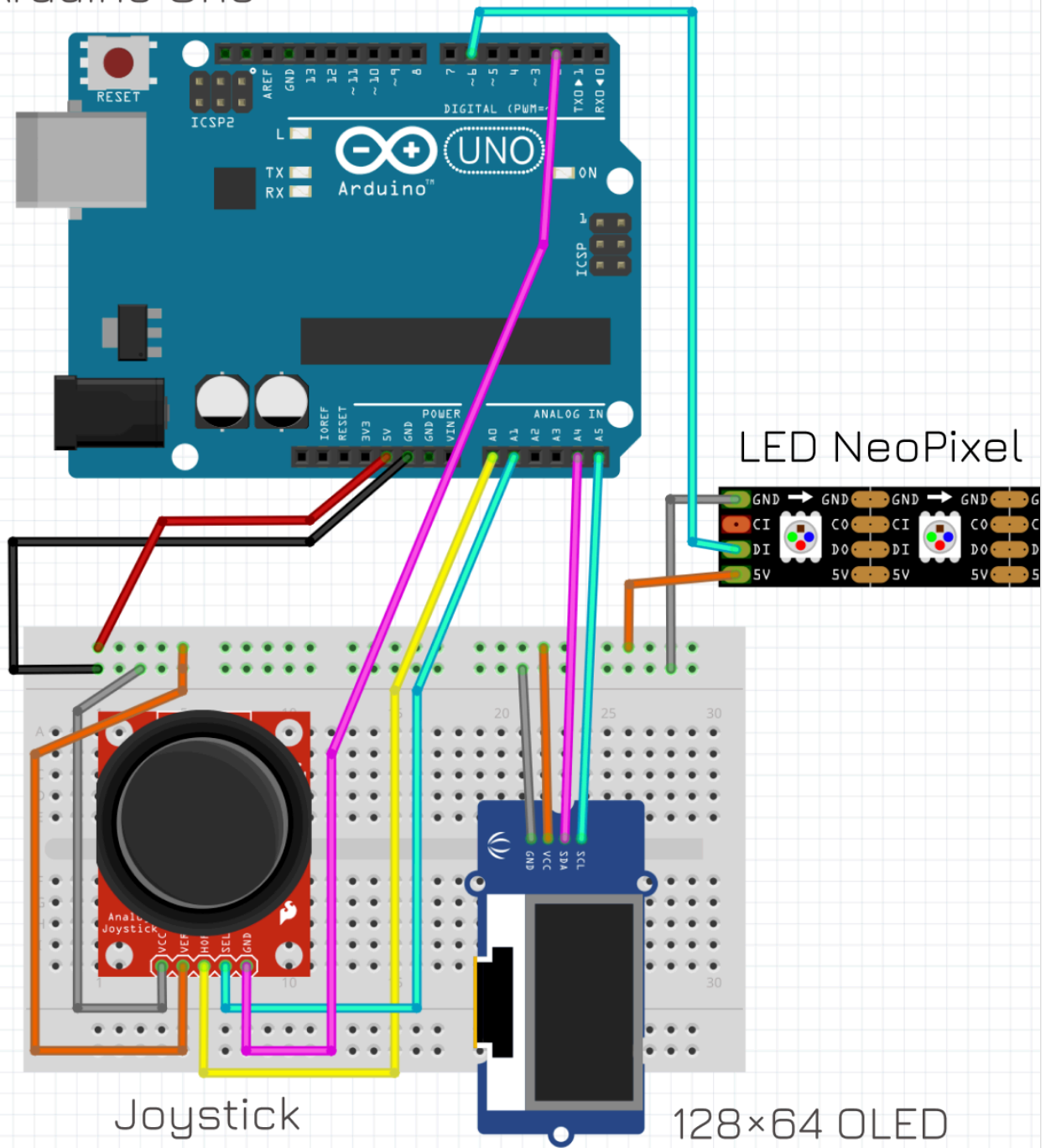
1. Identify the NeoPixel pins: GND, DIN, 5V
2. Connect the NeoPixel wires:

Connect GND from the NeoPixel strip to Arduino's GND pin

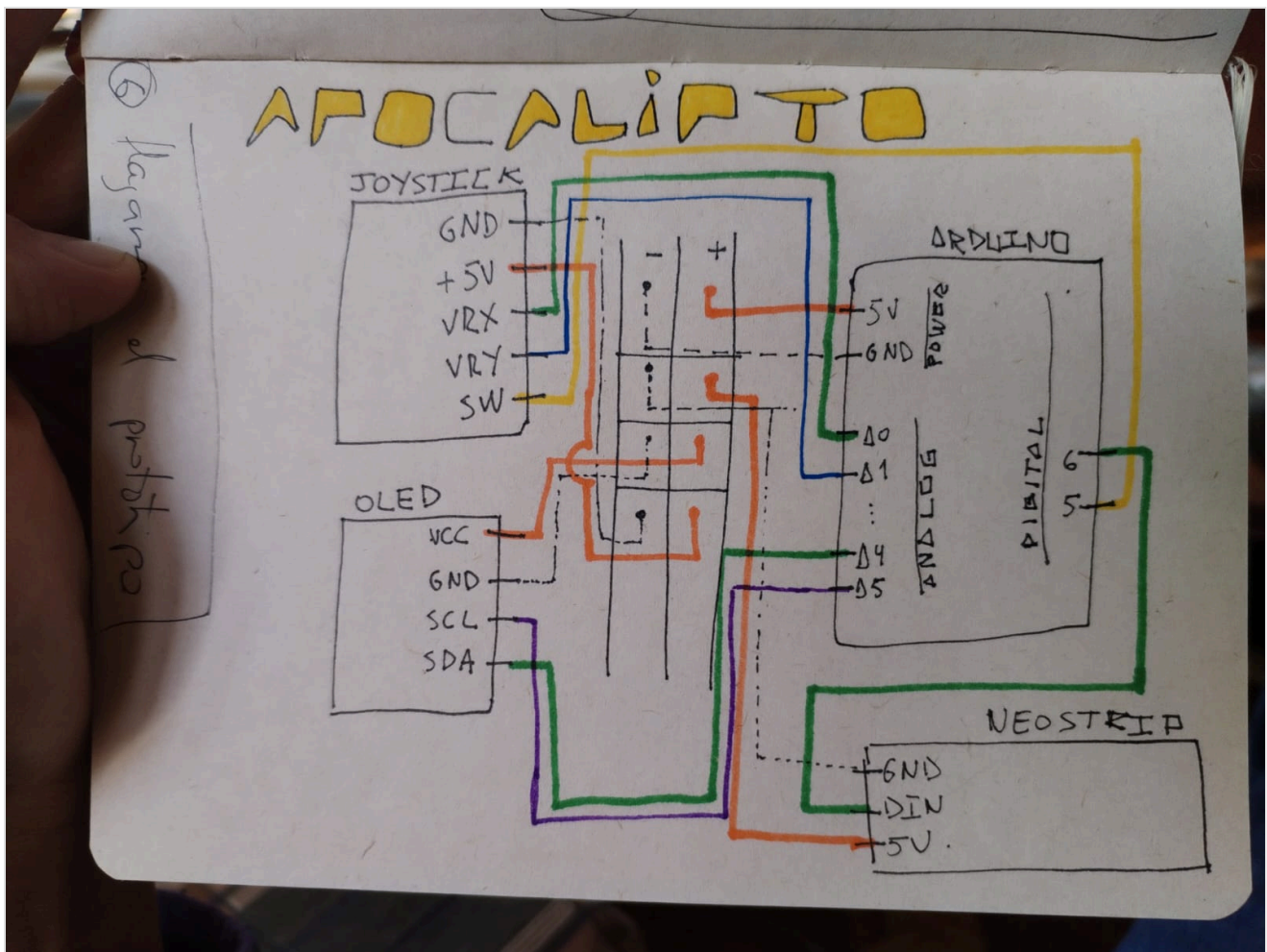
Connect DIN from the NeoPixel strip to Arduino's Pin 6

Connect 5V from the NeoPixel strip to Arduino's 5V pin

Arduino Uno



Full wiring diagram — joystick, OLED, and NeoPixel strip on the Arduino Uno.



Juan's original hand-drawn schematic — sketched before wiring.

SOFTWARE UPLOAD

1. Connect the Arduino to your computer via USB
2. Open the Arduino IDE
3. Install the required libraries, from Tools > Manage Libraries:
 - Adafruit NeoPixel
 - Adafruit GFX Library
 - Adafruit SSD1306
4. Open Demo Code (the .ino file) from the project repository
5. Select the correct board:
 - Tools > Board > Arduino Uno
6. Select the correct port:
 - Tools > Port > (select the new COM port)
7. Click the Upload button (arrow icon)

CODE HIGHLIGHTS

The complete Demo Code lives in this project's GitHub repository — saved there as its own file, so it isn't reproduced in full here. Below are the most meaningful pieces of it, each with a plain-language explanation of what it does and why it's built that way.

1. State machine

The game's structure is one enum (GameState) and one switch statement in loop(). Nothing runs outside of its assigned state — this keeps the logic easy to follow even as the story gets more elaborate.

```
enum GameState { KAY_PACHA, MAZE_PACHA, UKU_PACHA, HANAN_PACHA, HOLD_PACHA, RESTART_TRANSITION };
GameState state = KAY_PACHA;

void loop() {
  switch (state) {
    case KAY_PACHA:      loopKayPacha();      break;
    case MAZE_PACHA:     loopMazePacha();     break;
    case UKU_PACHA:      loopUkuPacha();      break;
    case HANAN_PACHA:    loopHananPacha();    break;
    case HOLD_PACHA:     loopHoldPacha();     break;
    case RESTART_TRANSITION: loopRestartTransition(); break;
  }
}
```

2. Analog input

The joystick reports two continuous values (0–1023), read exactly the way a potentiometer is read. Moving it up on the Y axis is what triggers a jump — a small, deliberate deviation from using the click button.

```
bool upNow = (y < 400);
if (upNow && !joyUpWasActive && onGround) {
  vy = JUMP_V;
  onGround = false;
}
```

3. Memory constraints

The Arduino Uno has only 2KB of RAM, and the OLED buffer alone uses about half of that. Because of this, the maze is hand-drawn once and stored in flash memory (PROGMEM) instead of being generated by an algorithm — a real recursive maze generator would risk the little stack memory the board has left.

```
const uint16_t PROGMEM mazeRows[8] = {
  0b0000010001000111,
  0b1111010101010111,
  // ...six more rows
};

bool isWallAt(int x, int y) {
  uint16_t row = pgm_read_word(&mazeRows[y]);
  return (row >> (15 - x)) & 1;
}
```

4. Timing without delay()

Instead of pausing the whole program between phases, the code checks millis() to know how much time has passed. This keeps the joystick responsive at every moment, even during timed sequences like the strobe or the glyph rain.

```
unsigned long elapsedPhase = millis() - stateStart;
float t = constrain((float)elapsedPhase / (float)FLASH_DURATION, 0.0, 1.0);
```

5. PROGMEM sprites

The curiingue's bitmap is stored in flash memory rather than RAM, and read back with `pgm_read_byte()` whenever it needs to be drawn. This is the same technique used for the maze layout, and it is what makes the sketch fit on the Uno at all.

```
const unsigned char* bmp = frameA ? curi_alert_a : curi_alert_b;
unsigned char b = pgm_read_byte(&bmp[byteIndex]);
if (b & (0x80 >> (sx % 8))) display.drawPixel(x + ox, y + oy, SSD1306_WHITE);
```

6. The hook for real sensor data

Hazards spawn faster the farther the curiingue has traveled — but that "distance" is just a stand-in. This function was written so a real water-quality sensor reading can replace it directly later, without touching anything else in the game.

```
unsigned long getSpawnInterval(float progress) {
    progress = constrain(progress, 0.0, 1.0);
    return map((long)(progress * 1000), 0, 1000, SPAWN_MAX, SPAWN_MIN);
}
```

HOW THIS PROGRAM WORKS

The full design (shown in the videos and the Game Flow above) runs as a state machine: at any moment, the program is in exactly one of six states, and only draws and updates whichever state is currently active. The Demo Code in this manual implements this same structure for the mountain descent (Kay Pacha) and the maze (Uku Pacha), plus supporting transition animations — the states below describe the complete design this project is built toward, fully realized once ported to the ESP32.

1. Kay Pacha — the descent. The curiingue walks toward the city, jumping over hazards.
2. Maze Pacha — the aerial maze, seen from above, searching for clean water.
3. Uku Pacha — the threshold. A strobing whiteout, triggered by either collapse or finding the water.
4. Hanan Pacha — the ascent. The bird rises, whole again.
5. Hold Pacha — a held breath at the top of the ascent.
6. Restart Transition — the glyph rain, before the cycle loops back to Kay Pacha.

Full source code (Demo Code), wiring diagrams, and updates: see the [project's GitHub repository](#).